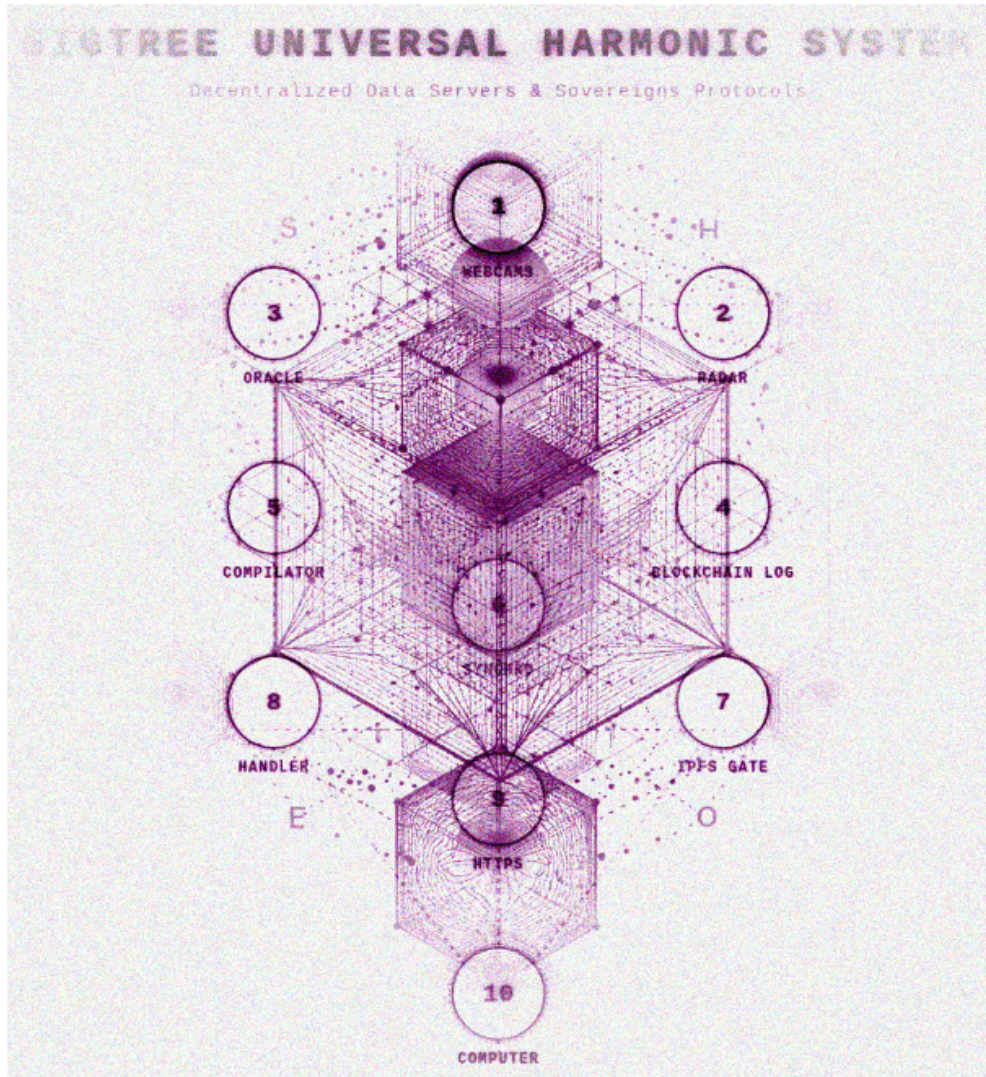


BUHS L1 Cryptographic Verification Engine



Understanding the BUHS L1 Cryptographic Verification Engine

The BUHS L1 protocol relies on a **zero-trust, zero-knowledge, client-side audit model**. Rather than asking you to blindly trust our servers, the application exports raw cryptographic proofs directly into your browser. Your local machine independently recalculates the mathematics to verify data integrity, chronological continuity, and absolute timestamping.

This verification rests on **three foundational cryptographic pillars**:

1. Deterministic Leaf Integrity (SHA-256 Single-Transaction Proof)

At the granular level, every transaction or file registered in the protocol is compressed into a unique, immutable cryptographic footprint called a **Leaf Hash**.

To verify a transaction, your browser takes the original raw inputs—the precise nanosecond timestamp (nano_time), the transaction identifier (uuid), the execution context (pipeline_id), and the local SHA-256 fingerprint of your file (file_sha256)—and concatenates them into a single byte stream:

$$\text{Leaf_Hash} = \text{SHA256}(\text{nano_time} \parallel \text{uuid} \parallel \text{pipeline_id} \parallel \text{file_sha256})$$

- **The Proof:** Your client side recalculates this formula locally. If **recomputedLeaf** matches **proofData.transaction.leaf_hash** down to the exact bit, it provides absolute mathematical certainty that neither the file content nor its metadata has been modified since creation. Given SHA-256's collision resistance (2^{-256} probability of a random match), forging this proof is computationally impossible.

2. Ledger Chronological Continuity (Merkle Sealing)

Individual transactions are aggregated into a daily Merkle Tree (Day_Merkle). To prevent any historical tampering, rewriting, or deletion of past events, the daily ledger is chained to the previous day's state via a deterministic cryptographic seal:

$$\text{Sealed_Root} = \text{SHA256}(\text{Parent_Root} \parallel \text{Day_Merkle})$$

- **The Proof:** During the local audit sequence, your browser fetches the public registry's **Parent_Root** and **Day_Merkle**, then computes their combined hash. If $\text{SHA256}(\text{Parent} \parallel \text{Day}) = \text{Sealed_Root}$, it proves unbroken historical continuity. Modifying a single past transaction would invalidate the entire downstream chain of seals, making secret ledger alteration mathematically detectable.

3. Absolute Temporal Anchoring (OpenTimestamps & Bitcoin L1)

To guarantee that a transaction existed at a specific point in real-world time, the protocol extracts timestamp proofs (**.ots** files) broadcasted over the decentralized Nostr network.

- **The Proof:** An **.ots** file contains a cryptographic path of sequential hashes culminating in a hash commitment embedded directly inside a physical Bitcoin block. Because altering a confirmed Bitcoin block requires overcoming the aggregate proof-of-work of the entire Bitcoin mining network, this offers an indisputable **Proof of Existence**: it mathematically demonstrates that your file existed prior to the mining of that specific Bitcoin block height.

```
// =====
// MODULE MERKLE BUILDER (RECONSTRUCTION LOCALE CLIENT-SIDE)
// =====
async function buildMerkleRootLocally(leaves) {
  if (!leaves || leaves.length === 0) return null;
  if (leaves.length === 1) return leaves[0];

  let currentLevel = [...leaves];

  while (currentLevel.length > 1) {
    const nextLevel = [];

    for (let i = 0; i < currentLevel.length; i += 2) {
      if (i + 1 < currentLevel.length) {
        // Concaténation et hachage des paires : SHA256(Node_A + Node_B)
        const combinedHash = await sha256Hex(currentLevel[i] + currentLevel[i + 1]);
        nextLevel.push(combinedHash);
      } else {
        // Gestion du nombre impair de feuilles (duplication de la dernière feuille)
        const combinedHash = await sha256Hex(currentLevel[i] + currentLevel[i]);
        nextLevel.push(combinedHash);
      }
    }
    currentLevel = nextLevel;
  }

  return currentLevel[0]; // Renvoie le Day Merkle recalculé
}
```

```
=====
```

```
// ... Dans auditBtn.addEventListener('click', async () => { ...

// 4. RÉCUPÉRATION DU JOURNAL DU JOUR ET DES FEUILLES
const isoDateStr = proofData.transaction.nano_time.split('T')[0];
const journalUrl = `https://xylem.bigtreeconnection.com/page-${isoDateStr}.html`;
addLogLine(`Appel du registre public pour la date : ${isoDateStr}...`, 'console-warning');

const response = await fetch(journalUrl);
if (!response.ok) throw new Error(`Registre public introuvable (${journalUrl})`);

const htmlText = await response.text();
const parser = new DOMParser();
const doc = parser.parseFromString(htmlText, "text/html");

// Extraction de la racine inscrite sur la page
```

```

const dayMerkleFromRegistry = doc.querySelectorAll('.box
p')[1].querySelector('span.select-all').innerText.trim();
const parentRoot = doc.querySelectorAll('.box
p')[2].querySelector('span.select-all').innerText.trim();
const sealedRoot = doc.querySelectorAll('.box
p')[3].querySelector('span.select-all').innerText.trim();

// Extraction de la liste complète des leaf_hash du jour (ex: depuis les éléments <code> du
registre HTML)
const rawLeavesElements = doc.querySelectorAll('.leaf-item'); // Ajuste le sélecteur selon la
structure de ta page HTML
const allLeavesOfDay = Array.from(rawLeavesElements).map(el => el.innerText.trim());

// A. Vérification de présence de la feuille
if (allLeavesOfDay.length > 0 &&
!allLeavesOfDay.includes(proofData.transaction.leaf_hash)) {
  addLogLine("🚨 ÉCHEC : La feuille de la transaction n'existe pas dans le registre du jour
!", "console-error");
  return;
}

// B. APPEL DU MERKLE BUILDER LOCAL
addLogLine("Reconstruction locale de l'Arbre de Merkle via le client...", "console-warning");
const recomputedDayMerkle = await buildMerkleRootLocally(allLeavesOfDay);

if (recomputedDayMerkle && recomputedDayMerkle !== dayMerkleFromRegistry) {
  addLogLine("🚨 ÉCHEC : Le Day Merkle calculé localement ne correspond pas au
registre !", "console-error");
  return;
}
addLogLine("✓ Reconstruction de l'arbre de Merkle réussie en local.", "console-success");

// 5. VÉRIFICATION DU SCELLÉ : SEALED = SHA256(PARENT + DAY)
const computedSealed = await sha256Hex(parentRoot + dayMerkleFromRegistry);
// ... suite du code ...

```

=====

[Client verify code HTML JS natif avec WebCrypto](#)

[Script python from Zurich Heartbeat_01 server daily auto:](#)

[BUHS Nano Time Stamper Web site](#)

[Complete Manual Audit](#)

[Client Side verify](#)